AI Assisted Coding

Lab Assignment – 13.1

Name: K. Sai Krishna Reddy

HT.No: 2303A52305

Batch: 45

Task Description #1 (Refactoring – Removing Code Duplication)

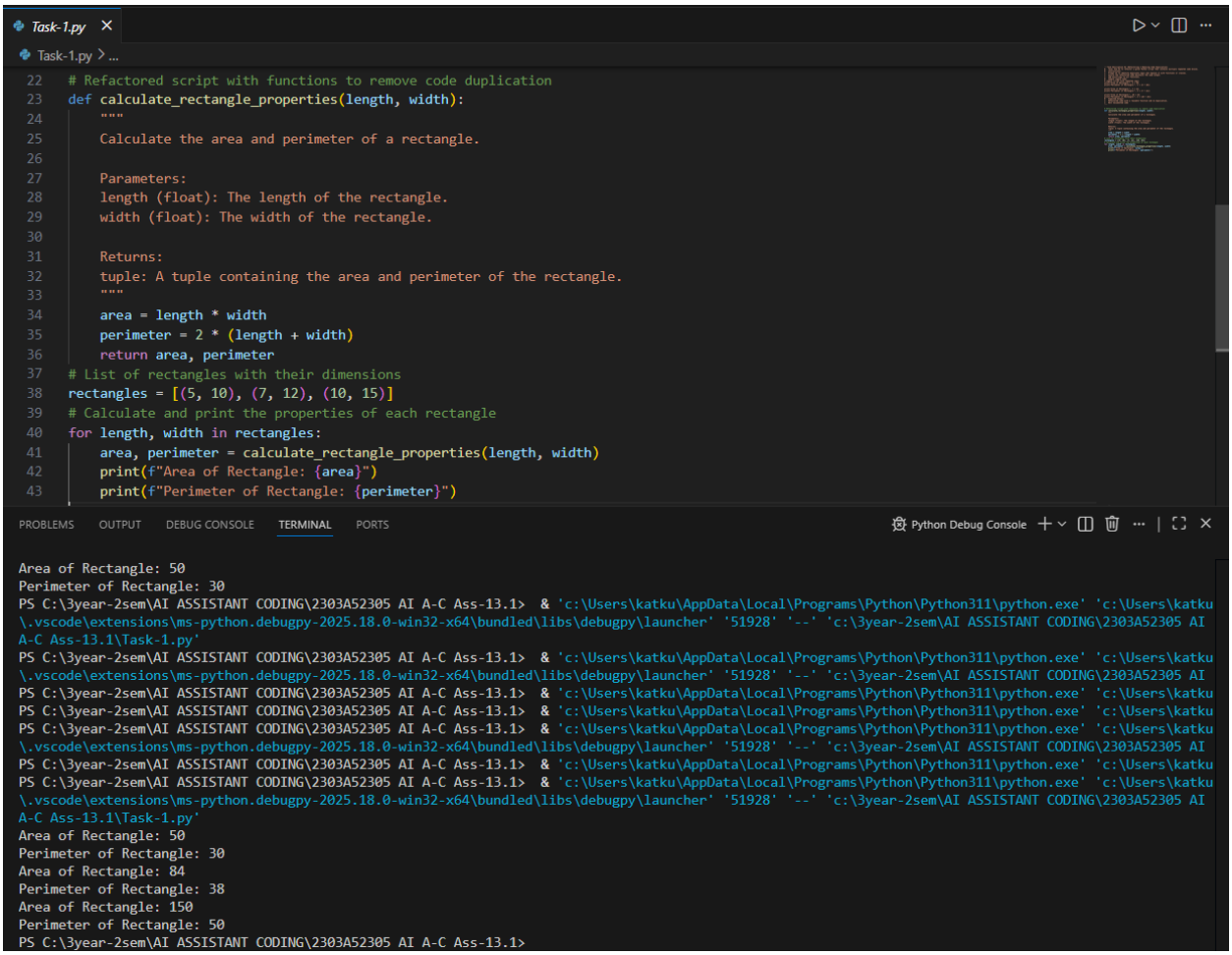
- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
    - Prompt AI to identify duplicate logic and replace it with functions or classes.
    - Ensure the refactored code maintains the same output.
    - Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
    - Refactored code with a reusable function and no duplication.
    - Well documented code



Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
    - Identify repeated or related logic and extract it into reusable functions.
    - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

Legacy script with inline repeated logic

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

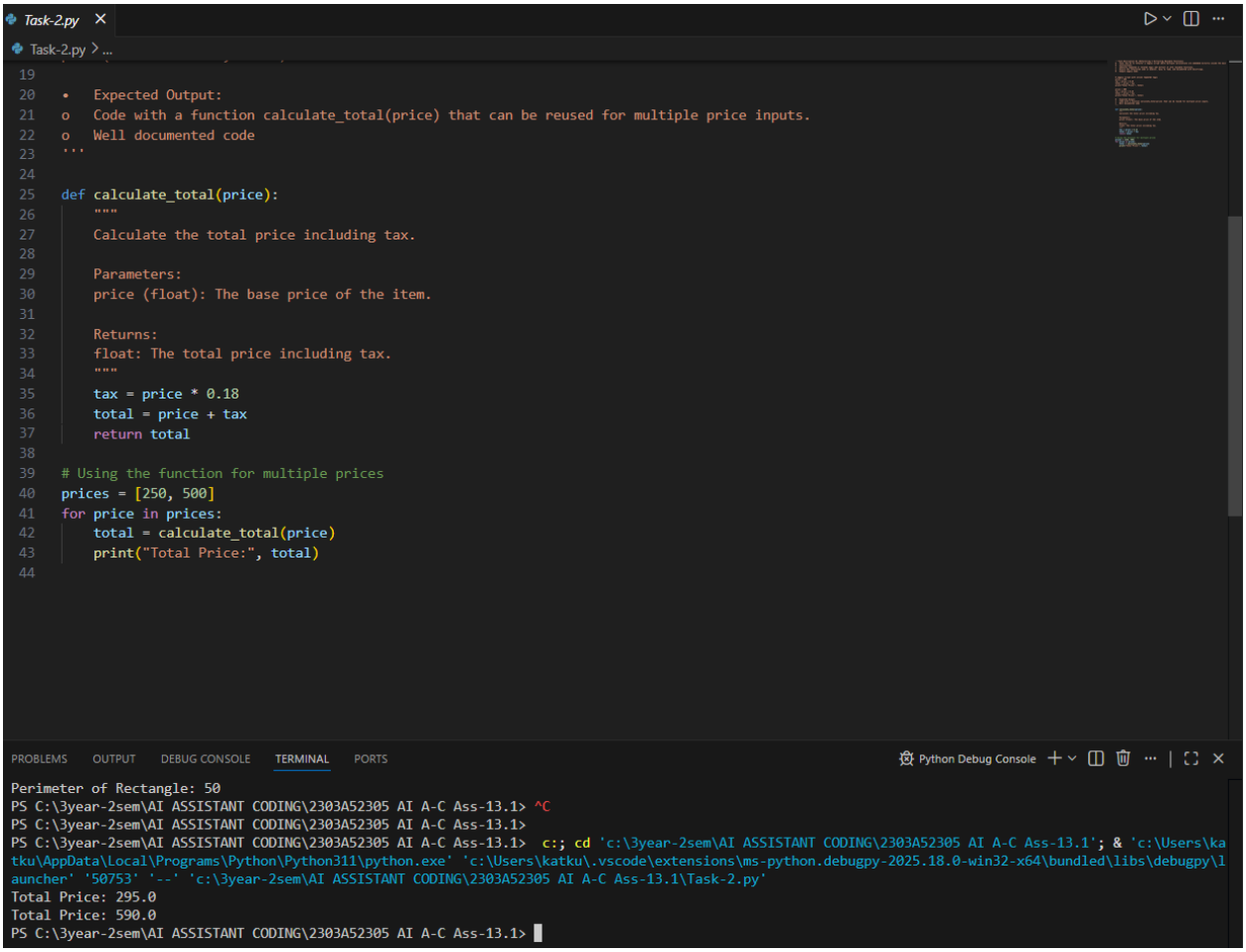
price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

- Expected Output:
    - Code with a function calculate_total(price) that can be reused for multiple price inputs.
    - Well documented code



Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
    - Accept marks as a parameter.
    - Return the corresponding grade as a string.
    - The grading logic must strictly follow the conditions below:
        - Marks ≥ 90 and ≤ 100 → "Grade A"
        - Marks ≥ 80 → "Grade B"
        - Marks ≥ 70 → "Grade C"
        - Marks ≥ 40 → "Grade D"
        - Marks ≥ 0 → "Fail"

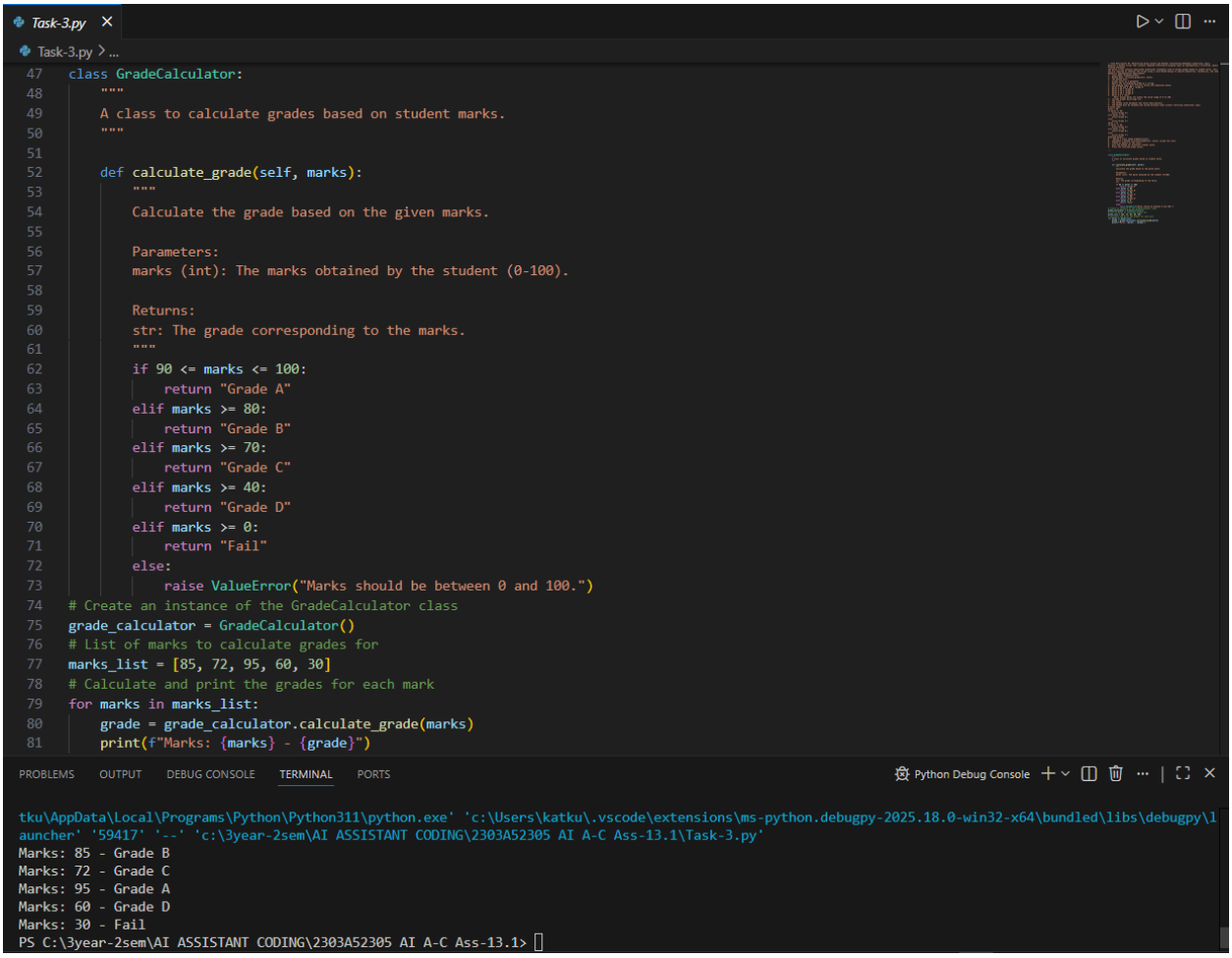
   Note: Assume marks are within the valid range of 0 to 100.
4. Include proper docstrings for:
    - The class
    - The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
   print("Grade A")
elif marks >= 75:
   print("Grade B")
else:
   print("Grade C")
marks = 72
if marks >= 90:
   print("Grade A")
elif marks >= 75:
   print("Grade B")
else:
   print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.



Task Description #4 (Refactoring – Converting Procedural Code to Functions)

• Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:
o Identify input, processing, and output sections.
o Convert each into a separate function.
o Improve code readability without changing behavior.

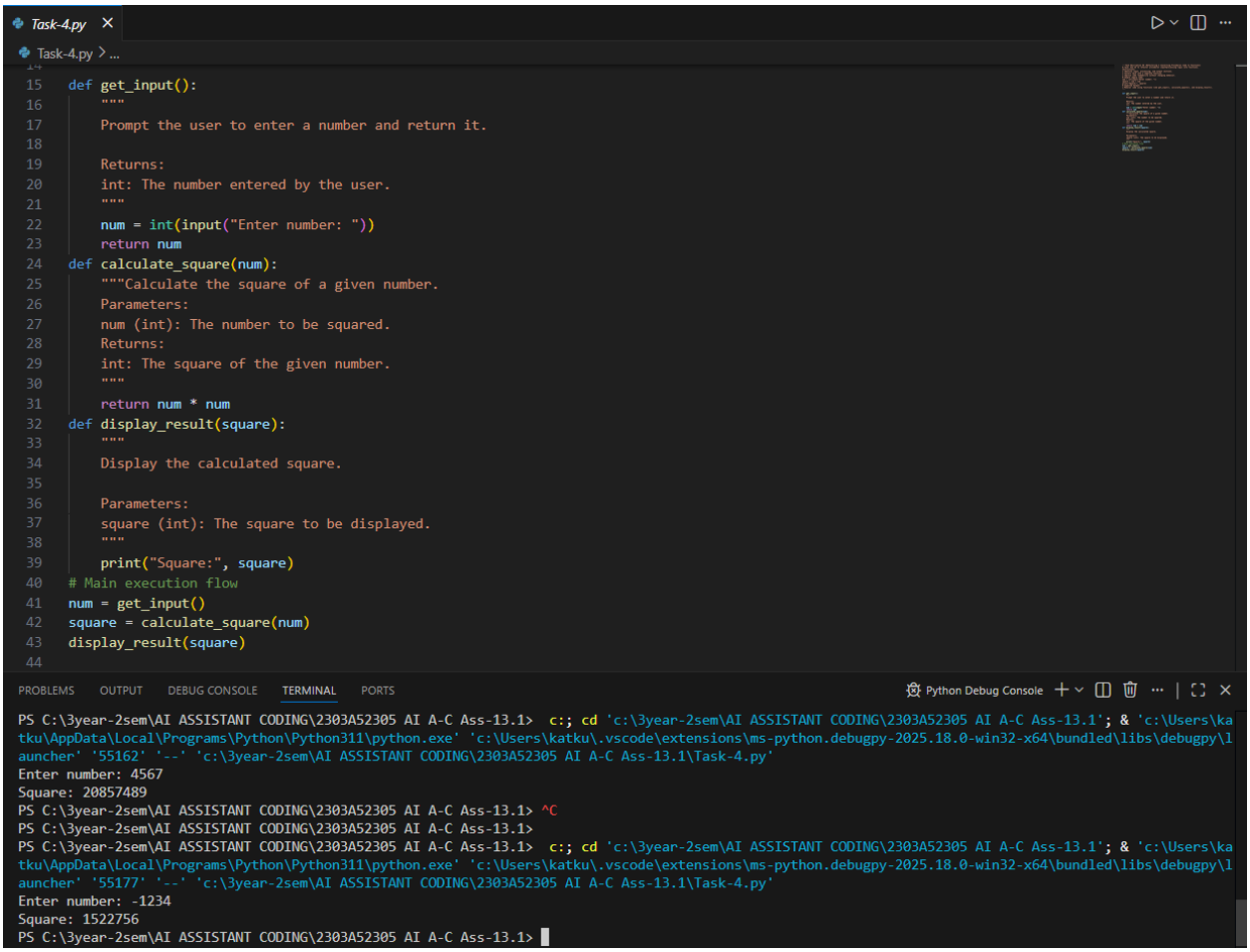
• Sample Legacy Code:

num = int(input("Enter number: "))

square = num * num

print("Square:", square)

• Expected Output:
o Modular code using functions like get_input(), calculate_square(), and display_result().



Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.
Focus Areas:
o Object-Oriented principles
o Encapsulation

Legacy Code:

salary = 50000

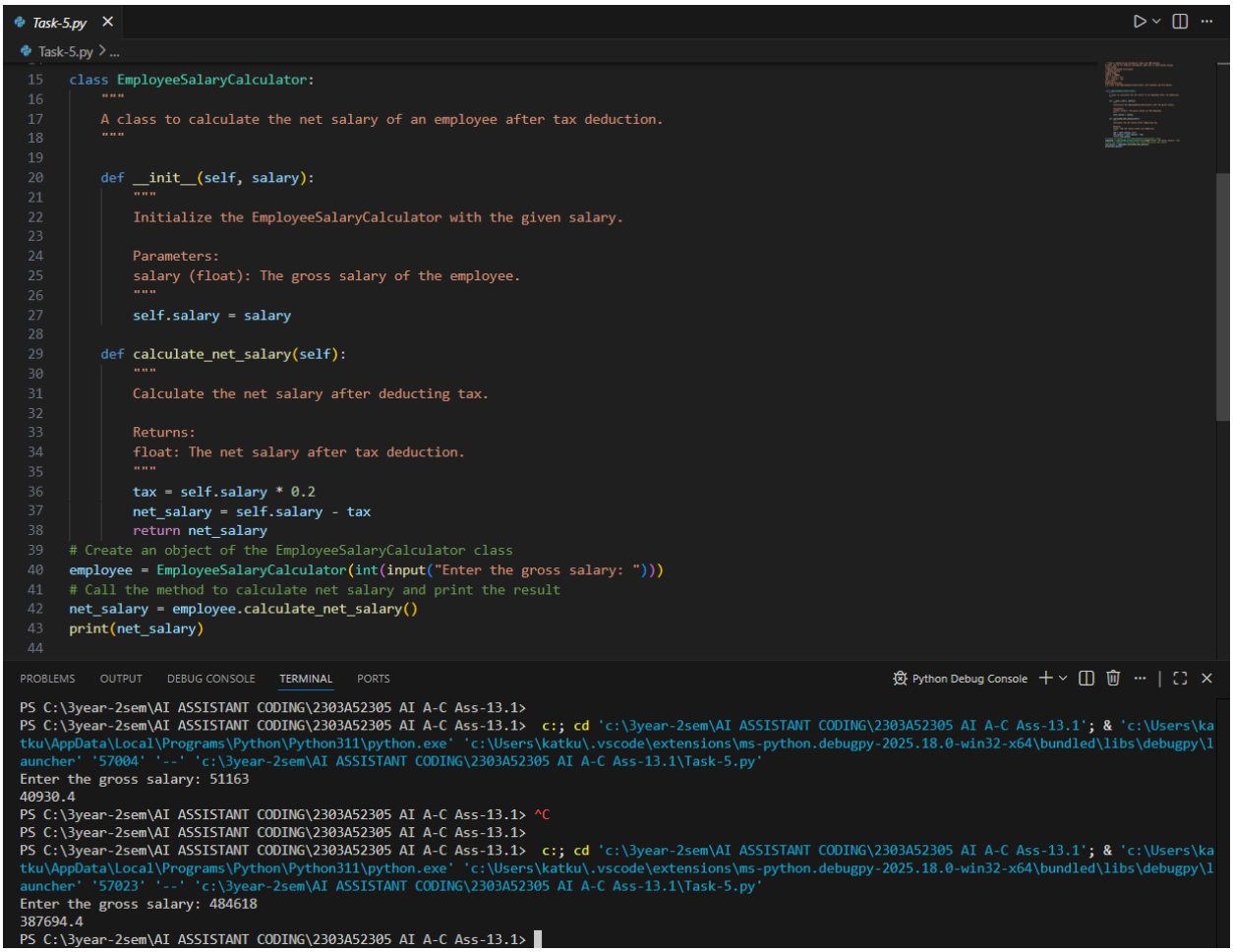
tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.



Task 6 (Optimizing Search Logic)

- **Task: Refactor inefficient linear searches using appropriate data structures.**
- Focus Areas:
  - o Time complexity
  - o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

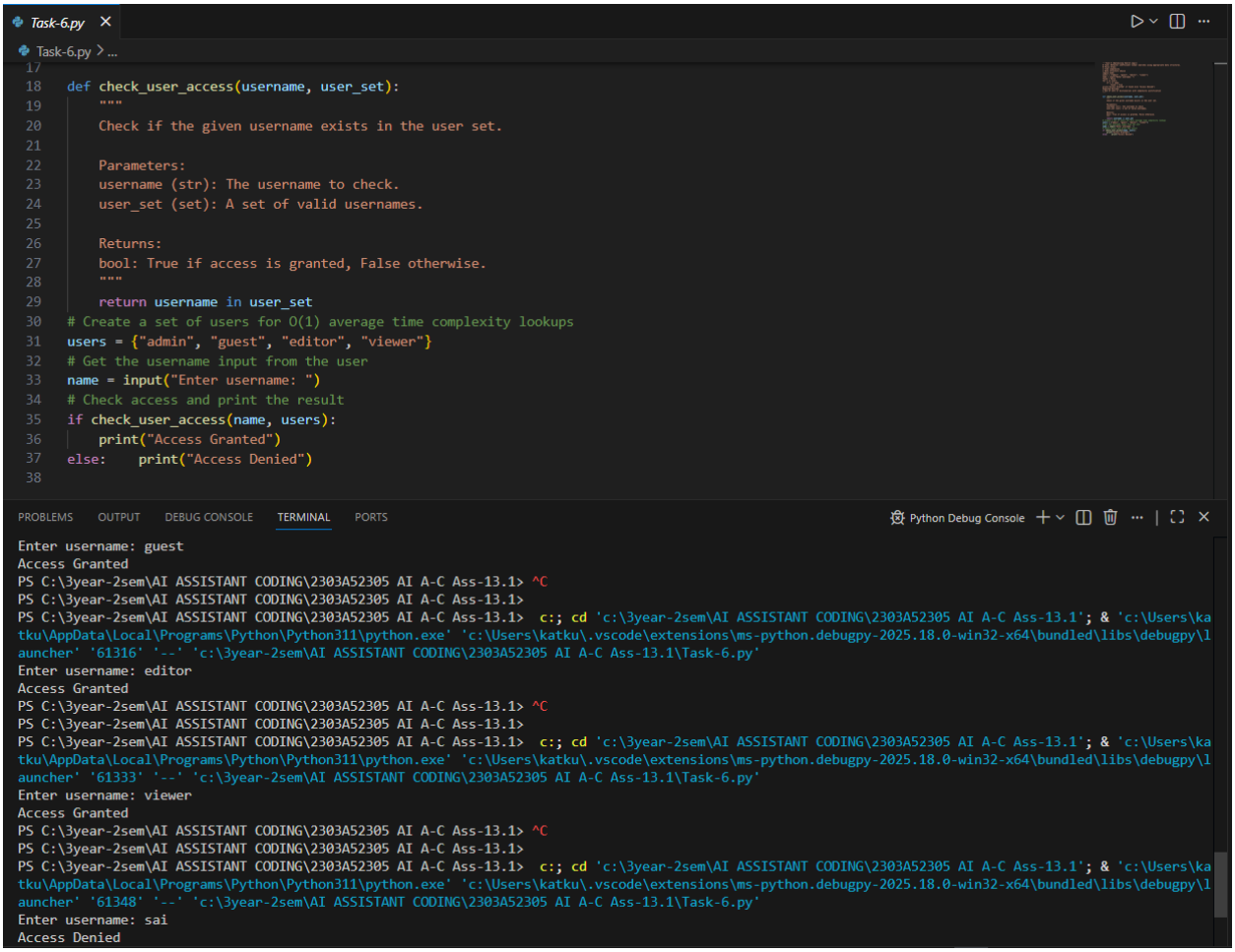
    if u == name:

        found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification



Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
    - add_book(title, author, isbn)
    - remove_book(isbn)
    - search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

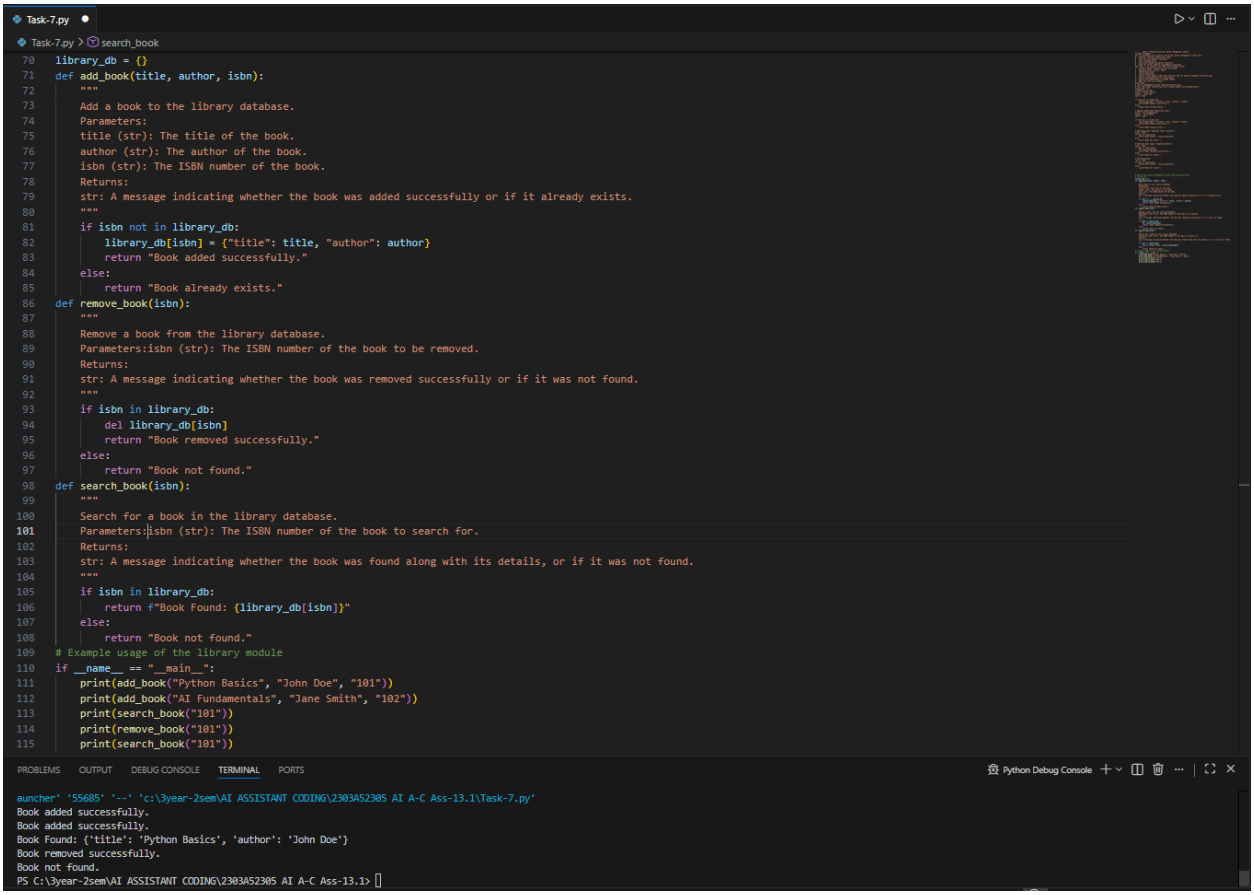
```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
```

```
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```



Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

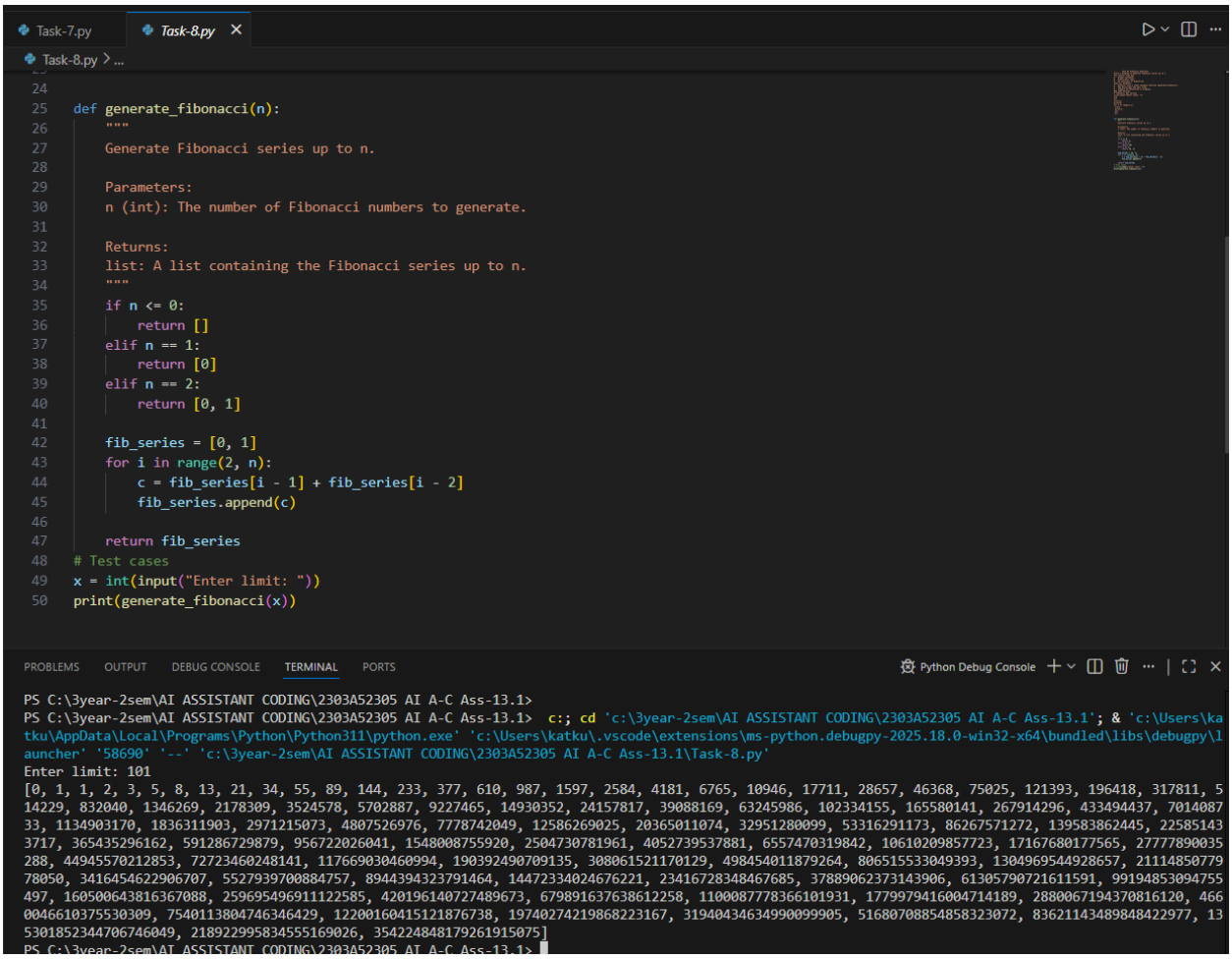
Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
```

```
for i in range(2,n):
 c=a+b
 print(c)
 a=b
b=c
```



Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

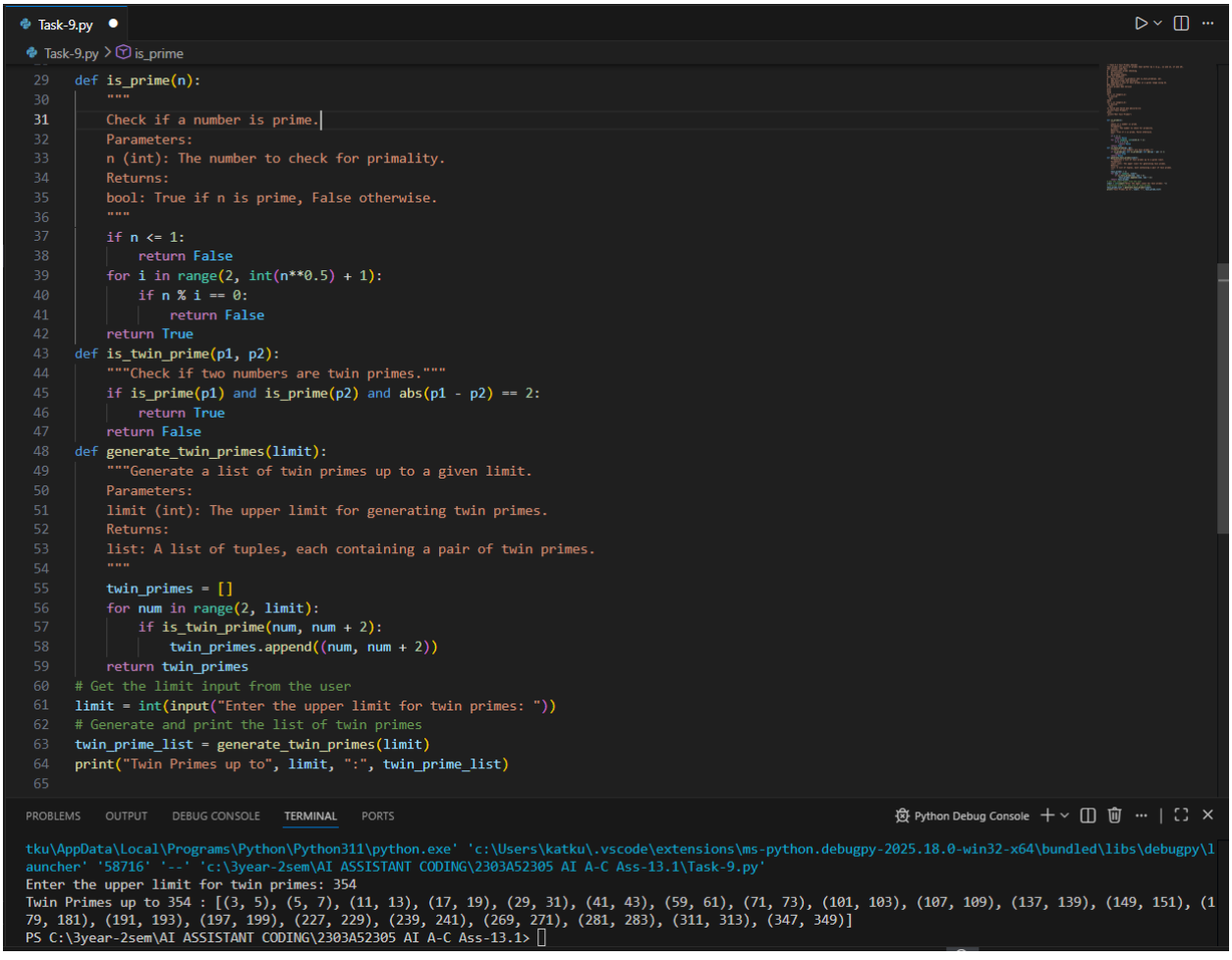
- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
 if a%i==0:
  fa=1
fb=0
for i in range(2,b):
 if b%i==0:
  fb=1
if fa==0 and fb==0 and abs(a-b)==2:
 print("Twin Primes")
else:
print("Not Twin Primes")
```



Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

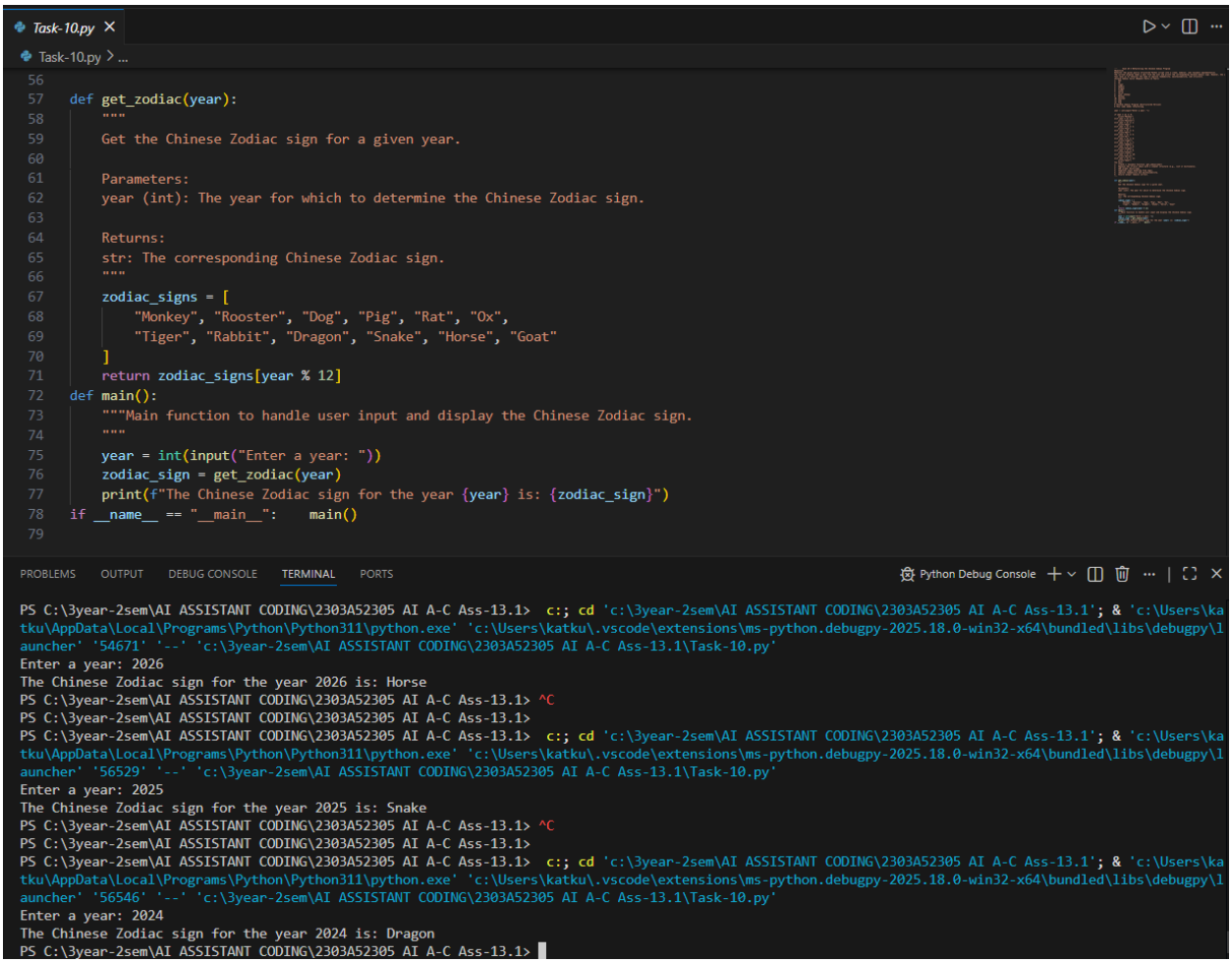
1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

```
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.
year = int(input("Enter a year: "))
if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
```

```
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.



```
Task-10.py

def get_zodiac(year):
    """
    Get the Chinese Zodiac sign for a given year.

    Parameters:
    year (int): The year for which to determine the Chinese Zodiac sign.

    Returns:
    str: The corresponding Chinese Zodiac sign.
    """
    zodiac_signs = [
        "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
        "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
    ]
    return zodiac_signs[year % 12]
def main():
    """Main function to handle user input and display the Chinese Zodiac sign.
    """
    year = int(input("Enter a year: "))
    zodiac_sign = get_zodiac(year)
    print(f"The Chinese Zodiac sign for the year {year} is: {zodiac_sign}")
if __name__ == "__main__":    main()

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS        Python Debug Console

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>  c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54671' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-10.py'
Enter a year: 2026
The Chinese Zodiac sign for the year 2026 is: Horse
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>  c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56529' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-10.py'
Enter a year: 2025
The Chinese Zodiac sign for the year 2025 is: Snake
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>  c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '56546' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-10.py'
Enter a year: 2024
The Chinese Zodiac sign for the year 2024 is: Dragon
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- **18 → 1 + 8 = 9 → 18 ÷ 9 = 2 ✅ (Harshad Number)**
- **19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer ❌ (Not Harshad)**

Problem Statement

The current implementation:

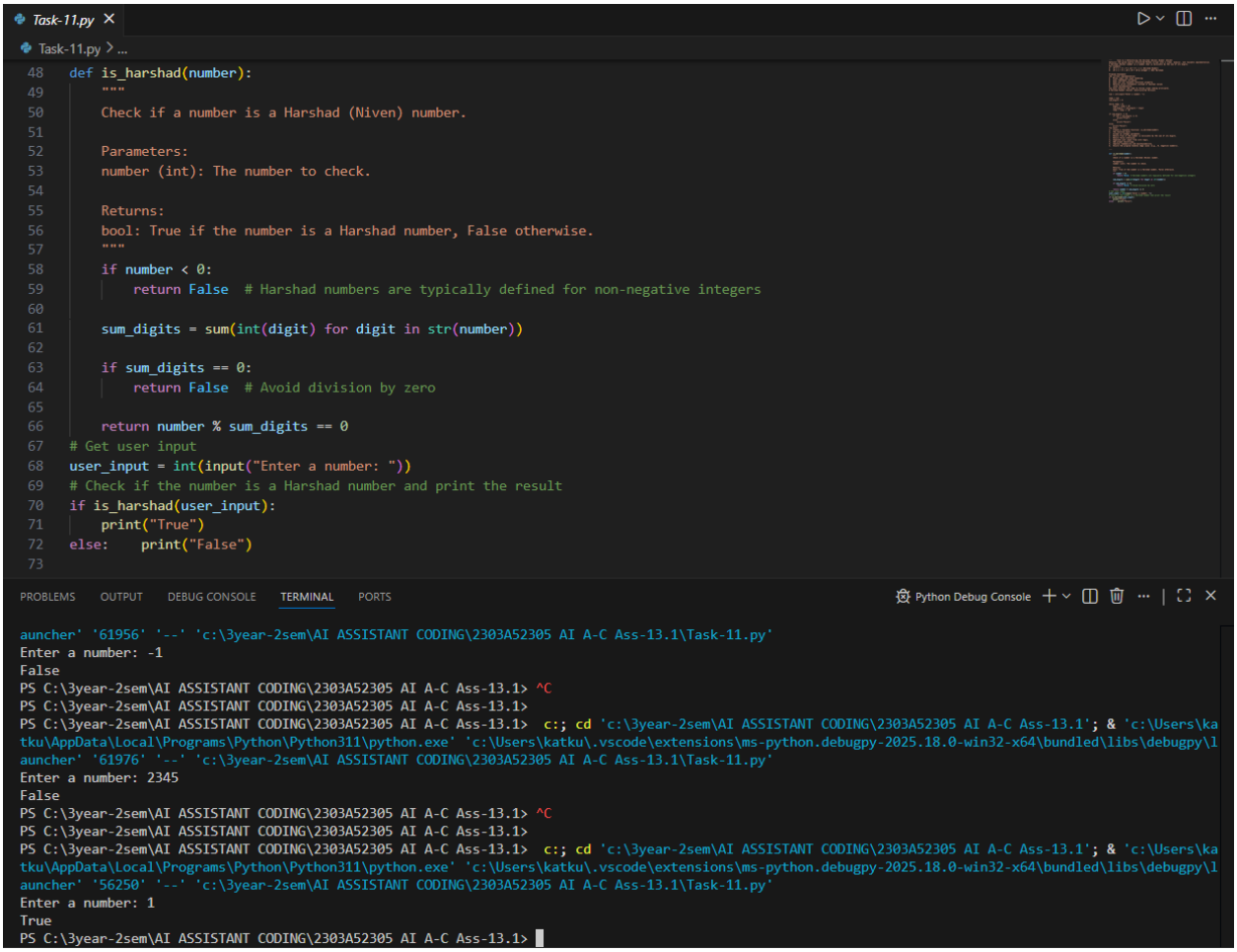
- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0
while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10
if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")
```

You must:

1. Create a reusable function: is_harshad(number)
2. The function must:
    - Accept an integer parameter.
    - Return True if the number is divisible by the sum of its digits.
    - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).



Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

 # Factorial Trailing Zeros (Unstructured Version)

n = int(input("Enter a number: "))

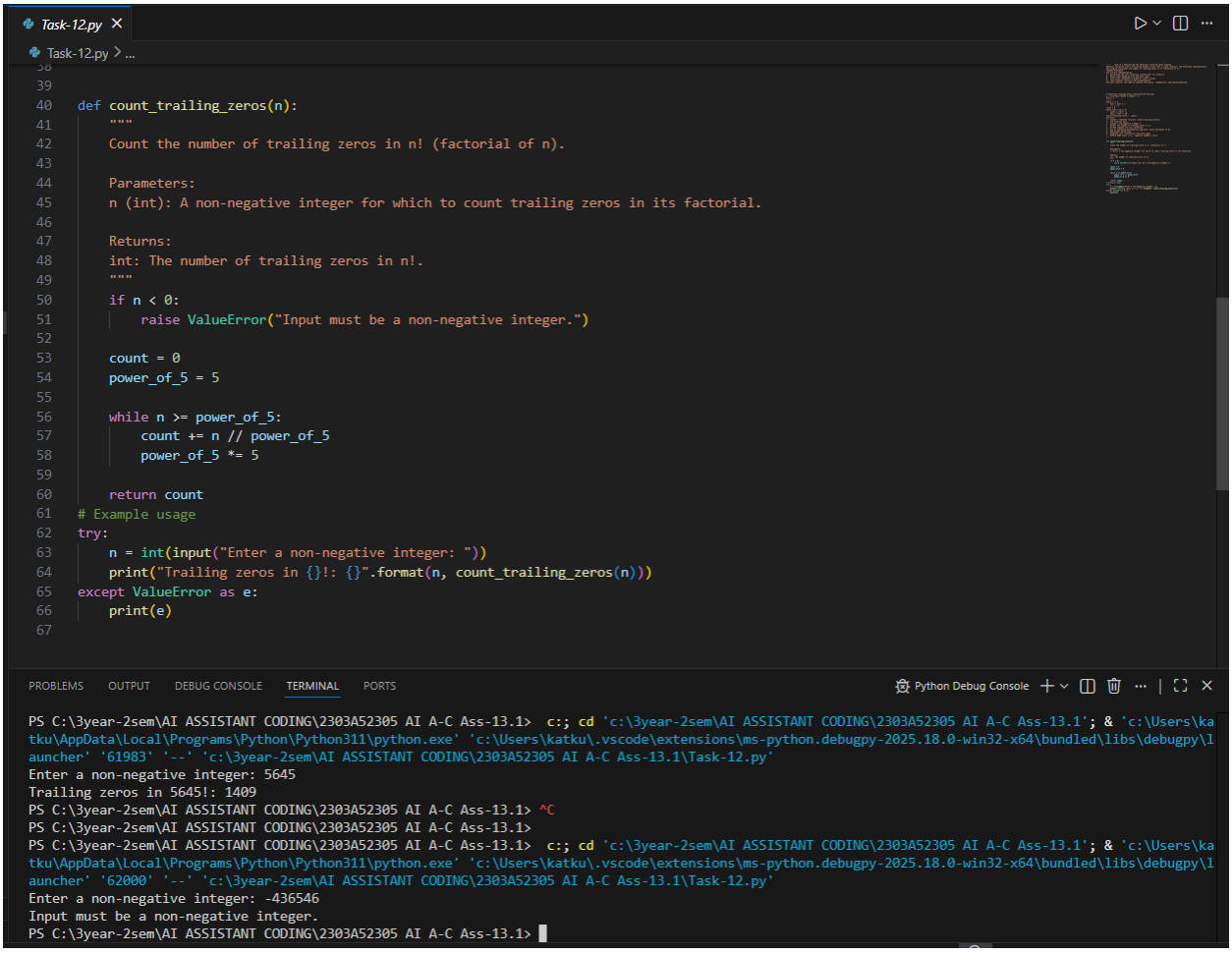
fact = 1

i = 1

```
while i <= n:
    fact = fact * i
    i = i + 1
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
    - Accept a non-negative integer n.
    - Return the number of trailing zeros in **n!**.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).



Task 13 (Collatz Sequence Generator – Test Case Design)

• Function: Generate Collatz sequence until reaching 1.

• Test Cases to Design:

- Normal: 6 → [6,3,10,5,16,8,4,2,1]
- Edge: 1 → [1]
- Negative: -5
- Large: 27 (well-known long sequence)

• Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the 3n+1 sequence).
- The rules are:
  - If n is even → next = n / 2.
  - If n is odd → next = 3n + 1.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even → 6/2 = 3)
- 3 (odd → 3*3+1 = 10)
- 10 (even → 10/2 = 5)
- 5 (odd → 3*5+1 = 16)
- 16 (even → 16/2 = 8)
- 8 (even → 8/2 = 4)
- 4 (even → 4/2 = 2)
- 2 (even → 2/2 = 1)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

Task-13.py

```python
def collatz_sequence(n):
    """
    Generate the Collatz sequence for a given integer n until reaching 1.

    Parameters:
    n (int): The starting integer for the Collatz sequence.

    Returns:
    list: A list containing the Collatz sequence from n to 1.
    """
    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []

    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1

    sequence.append(1)  # Append the final element of the sequence
    return sequence
# Example usage
try:
    n = int(input("Enter a positive integer: "))
    print(collatz_sequence(n))
except ValueError as e:
    print(e)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python Debug Console

```
tku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '59936' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-13.py'
Enter a positive integer: 866
[866, 433, 1300, 650, 325, 976, 488, 244, 122, 61, 184, 92, 46, 23, 70, 35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>  c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\ka
tku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\l
auncher' '59951' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-13.py'
Enter a positive integer: 2222
[2222, 1111, 3334, 1667, 5002, 2501, 7504, 3752, 1876, 938, 469, 1408, 704, 352, 176, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1
]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

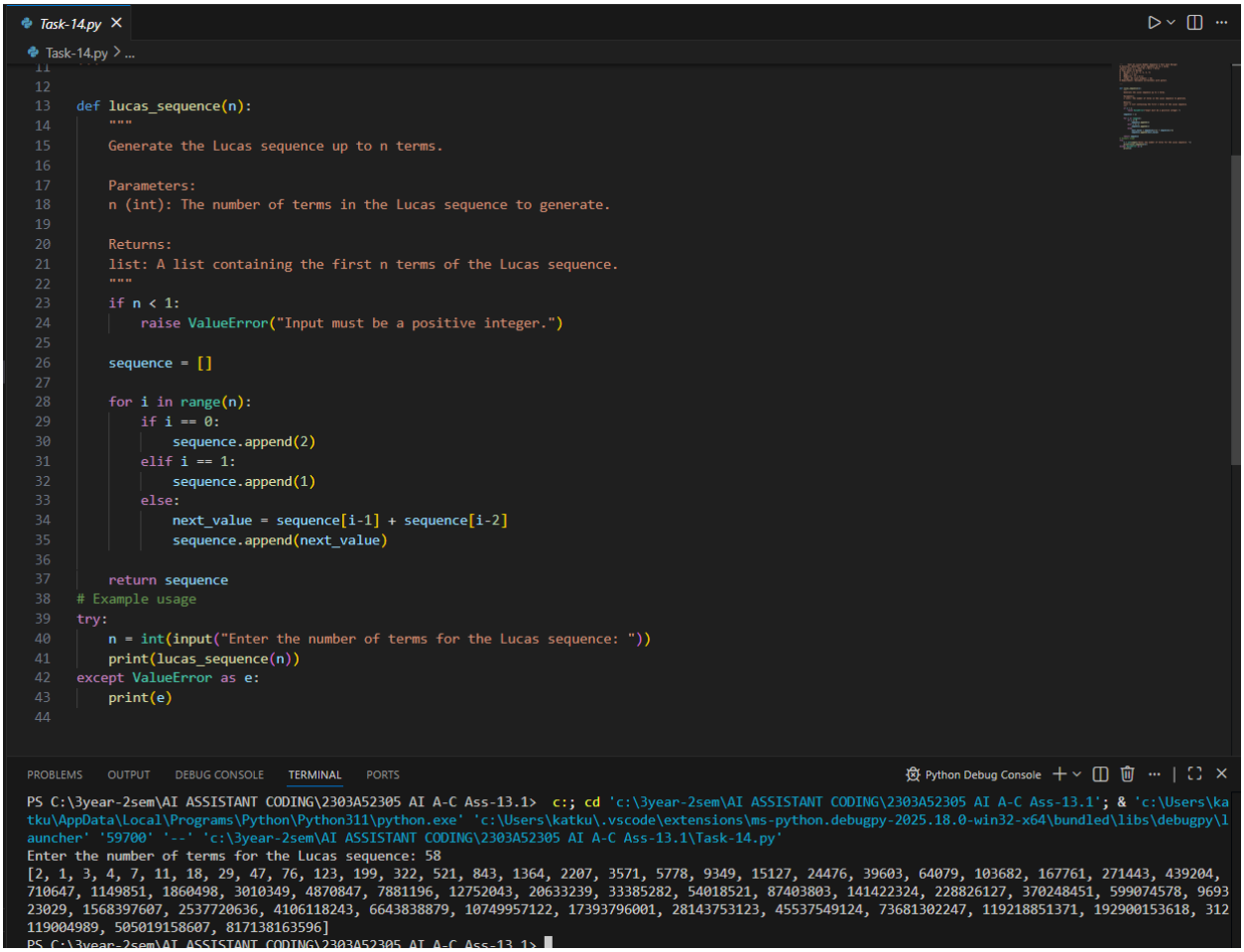
Task 14 (Lucas Number Sequence – Test Case Design)

• Function: Generate Lucas sequence up to n terms.
(Starts with 2,1, then Fn = Fn-1 + Fn-2)

• Test Cases to Design:

- Normal: 5 → [2, 1, 3, 4, 7]
- Edge: 1 → [2]
- Negative: -5 → Error
- Large: 10 (last element = 76).

• Requirement: Validate correctness with pytest.



```python
def lucas_sequence(n):
    """
    Generate the Lucas sequence up to n terms.

    Parameters:
    n (int): The number of terms in the Lucas sequence to generate.

    Returns:
    list: A list containing the first n terms of the Lucas sequence.
    """
    if n < 1:
        raise ValueError("Input must be a positive integer.")

    sequence = []

    for i in range(n):
        if i == 0:
            sequence.append(2)
        elif i == 1:
            sequence.append(1)
        else:
            next_value = sequence[i-1] + sequence[i-2]
            sequence.append(next_value)

    return sequence
# Example usage
try:
    n = int(input("Enter the number of terms for the Lucas sequence: "))
    print(lucas_sequence(n))
except ValueError as e:
    print(e)
```

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>  c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59700' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1\Task-14.py'
Enter the number of terms for the Lucas sequence: 58
[2, 1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322, 521, 843, 1364, 2207, 3571, 5778, 9349, 15127, 24476, 39603, 64079, 103682, 167761, 271443, 439204, 710647, 1149851, 1860498, 3010349, 4870847, 7881196, 12752043, 20633239, 33385282, 54018521, 87403803, 141422324, 228826127, 370248451, 599074578, 969323029, 1568397607, 2537720636, 4106118243, 6643838879, 10749957122, 17393796001, 28143753123, 45537549124, 73681302247, 119218851371, 192900153618, 312119004989, 505019158607, 817138163596]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.1>
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

• Function: Count vowels and consonants in string.

• Test Cases to Design:

- Normal: "hello" → (2,3)
- Edge: "" → (0,0)
- Only vowels: "aeiou" → (5,0)

Large: Long text

• Requirement: Validate correctness with pytest.